

Appendix - Supporting Figures and Source Code

Syed Bashir Hydari et al.

2026-05-06

Table of contents

1	Overview	2
2	Setup	2
3	Figure A1 — Monte Carlo Wealth Trajectories (§3.1)	3
4	Figure A2 — Final Wealth Distributions Across Horizons (§3.1)	6
5	Figure A3 — Banca Intesa Cumulative Wealth (§3.2)	8
6	Figure A4 — Mean-Variance Efficient Frontier, In-Sample EuroStoxx 50 (§3.3)	11
7	Figure A5 — Long-Only OOS Cumulative Returns, EuroStoxx 50 (§3.4)	16
8	Figure A6 — HAC Pairwise Return Differences (§3.5)	19
9	Figure A7 — Unconstrained Allocation Stack, Half Kelly (§3.6)	21

1 Overview

This appendix presents seven supporting figures referenced in the main report alongside the executable source code that produces each one. Section 2 sets up the global R environment. Sections 3–9 present Figures A1 through A7, one section each, with the figure rendered live from the code shown. Each figure is anchored to a specific subsection of the main report’s results: A1 and A2 to §3.1 (Monte Carlo), A3 to §3.2 (Banca Intesa), A4 to §3.3 (in-sample EuroStoxx), A5 to §3.4 (long-only OOS), A6 to §3.5 (HAC inference), and A7 to §3.6 (unconstrained OOS). The pipeline downloads market data from Yahoo Finance via `tidyquant/quantmod` and runs all rolling-window backtests in-document.

2 Setup

The block below loads the package stack used throughout the appendix and applies a namespace fix that forces `dplyr`’s `filter`, `lag`, and `select` to win over the masking versions exported by `xts`, `stats`, and `PerformanceAnalytics`. Without this, rolling-window operations elsewhere in the pipeline silently use the wrong `lag()` and produce subtly corrupted return series.

```
suppressPackageStartupMessages({
  library(tidyverse)
  library(scales)
  library(patchwork)
  library(knitr)
  library(quantmod)
  library(tidyquant)
  library(xts)
  library(zoo)
  library(quadprog)
  library(Matrix)
  library(PerformanceAnalytics)
  library(portfolioBacktest)
  library(sandwich)
  library(lmtest)
})

# Force dplyr verbs to win over xts/stats/PerformanceAnalytics masks.
filter <- dplyr::filter
lag     <- dplyr::lag
select <- dplyr::select

options(scipen = 999)
options(knitr.kable.NA = "")
theme_set(theme_minimal(base_size = 11))

GLOBAL_SEED <- 123
set.seed(GLOBAL_SEED)
```

3 Figure A1 — Monte Carlo Wealth Trajectories (§3.1)

Sample wealth trajectories under each Kelly multiple over 1,000 simulated trades. The visual signature of the volatility-pumping geometry discussed in §3.1 of the main report: Half Kelly's paths fan upward in a tight band, Full Kelly's paths spread but stay broadly positive, and Double / Triple Kelly's paths exhibit the lottery-like dispersion that drives the large mean / collapsed median pattern in Table 1. Market-calibrated parameters follow CC20 §2.3: $\mu_{\text{ann}} = 0.12$, $\sigma_{\text{ann}} = 0.40$, $r_f = 0.01$, with theoretical full Kelly ≈ 0.6875 .

```
MU_ANNUAL    <- 0.12
SIGMA_ANNUAL <- 0.40
RF_ANNUAL    <- 0.01
TRADING_DAYS <- 252

MU_DAILY     <- MU_ANNUAL / TRADING_DAYS
SIGMA_DAILY  <- SIGMA_ANNUAL / sqrt(TRADING_DAYS)
RF_DAILY     <- RF_ANNUAL / TRADING_DAYS

F_FULL_KELLY <- (MU_DAILY - RF_DAILY) / (SIGMA_DAILY^2)
F_HALF_KELLY <- 0.5 * F_FULL_KELLY
F_DOUBLE_KELLY <- 2.0 * F_FULL_KELLY
F_TRIPLE_KELLY <- 3.0 * F_FULL_KELLY

generate_gbm_returns <- function(n_periods, mu, sigma) {
  Z <- rnorm(n_periods)
  (mu - sigma^2/2) + sigma * Z
}

simulate_wealth <- function(W0, f, returns, rf) {
  n <- length(returns)
  W <- numeric(n + 1)
  W[1] <- W0
  for (t in 1:n) {
    W[t+1] <- W[t] * (1 - f) * exp(rf) + W[t] * f * exp(returns[t])
  }
  W
}

set.seed(456)
strategies_mc <- list(
  "Half Kelly"    = F_HALF_KELLY,
  "Full Kelly"    = F_FULL_KELLY,
  "Double Kelly"  = F_DOUBLE_KELLY,
  "Triple Kelly"  = F_TRIPLE_KELLY
)

path_data <- list()
```

```

for (sn in names(strategies_mc)) {
  f <- strategies_mc[[sn]]
  for (i in 1:30) {
    returns <- generate_gbm_returns(1000, MU_DAILY, SIGMA_DAILY)
    W <- simulate_wealth(1, f, returns, RF_DAILY)
    path_data[[paste(sn, i)]] <- tibble(
      strategy = sn, trajectory = i,
      period = 0:1000, wealth = W
    )
  }
}
paths_df <- bind_rows(path_data) %>%
  mutate(strategy = factor(strategy,
    levels = c("Half Kelly", "Full Kelly",
      "Double Kelly", "Triple Kelly")))

kelly_palette <- c("Half Kelly" = "#2ECC71",
  "Full Kelly" = "#E74C3C",
  "Double Kelly" = "#F39C12",
  "Triple Kelly" = "#9B59B6")

ggplot(paths_df, aes(x = period, y = wealth,
  group = interaction(strategy, trajectory),
  color = strategy)) +
  geom_line(alpha = 0.3, linewidth = 0.5) +
  facet_wrap(~ strategy, ncol = 2, scales = "free_y") +
  scale_y_log10(labels = number_format(accuracy = 0.1)) +
  scale_color_manual(values = kelly_palette) +
  labs(title = "Sample Wealth Paths (1,000 trades, 30 trajectories each)",
    x = "Trading Period", y = "Wealth (log scale)", color = "Strategy") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
    legend.position = "bottom",
    strip.text = element_text(face = "bold", size = 12))

```

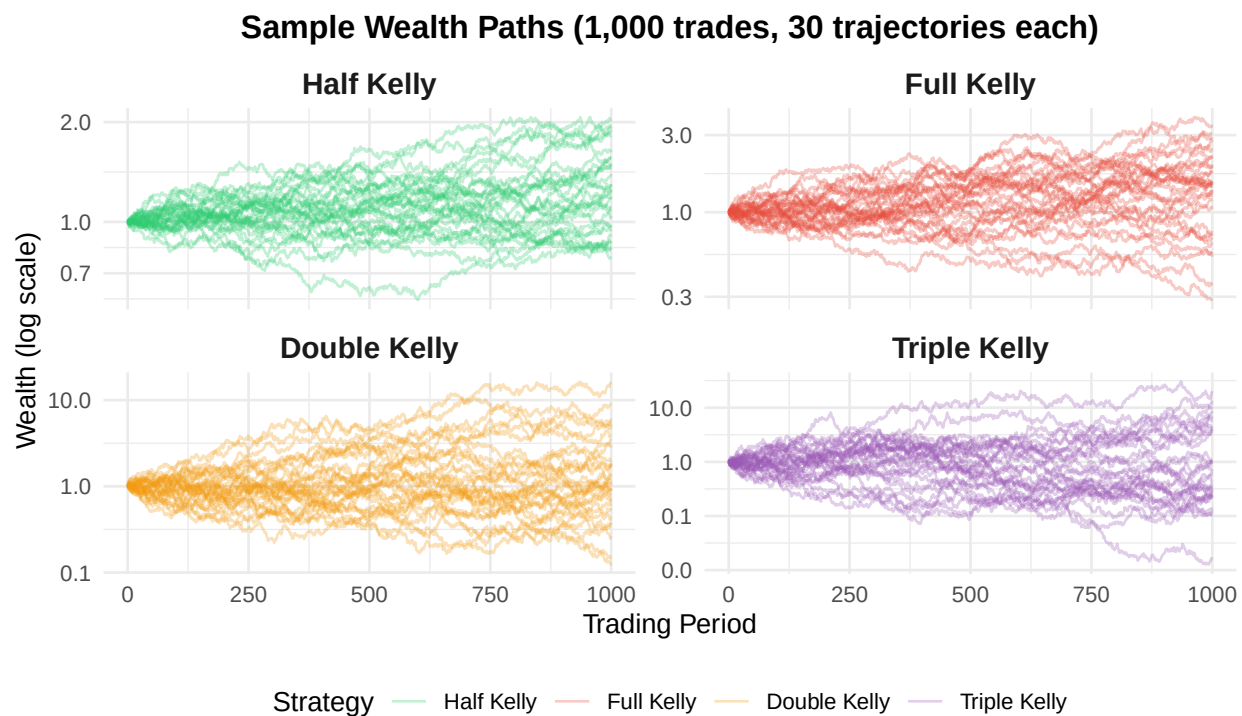


Figure A1: Sample wealth trajectories for Half, Full, Double, and Triple Kelly over 1,000 simulated trades (30 paths each).

4 Figure A2 — Final Wealth Distributions Across Horizons (§3.1)

Density estimates of terminal wealth at 100, 1,000, and 10,000 simulated trades, with dashed lines marking medians. This figure operationalizes the mean-vs-median story: at 10,000 trades, Triple Kelly's distribution (purple) develops a thick left tail at zero and a thin extreme right tail, while Half Kelly (green) remains tightly concentrated. The median collapse from ≈ 4.5 for Half Kelly to ≈ 0.01 for Triple Kelly visible here is the same effect Table 1 of the main report tabulates numerically.

```
plot_wealth_distribution <- function(n_trades, n_trajectories = 1000) {
  wealth_data <- list()
  for (sn in names(strategies_mc)) {
    f <- strategies_mc[[sn]]
    fw <- numeric(n_trajectories)
    for (i in 1:n_trajectories) {
      returns <- generate_gbm_returns(n_trades, MU_DAILY, SIGMA_DAILY)
      W <- simulate_wealth(1, f, returns, RF_DAILY)
      fw[i] <- W[length(W)]
    }
    wealth_data[[sn]] <- tibble(strategy = sn, final_wealth = fw)
  }
  wealth_df <- bind_rows(wealth_data) %>%
    mutate(strategy = factor(strategy,
                             levels = c("Half Kelly", "Full Kelly",
                                           "Double Kelly", "Triple Kelly")))

  ggplot(wealth_df, aes(x = final_wealth, fill = strategy, color = strategy)) +
    geom_density(alpha = 0.3, linewidth = 1) +
    geom_vline(data = wealth_df %>% group_by(strategy) %>%
               summarise(median = median(final_wealth)),
               aes(xintercept = median, color = strategy),
               linetype = "dashed", linewidth = 1) +
    scale_x_log10(labels = number_format(accuracy = 0.1)) +
    scale_fill_manual(values = kelly_palette) +
    scale_color_manual(values = kelly_palette) +
    labs(title = sprintf("%d trades", n_trades),
         x = "Final Wealth (log scale)", y = "Density") +
    theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 11),
          legend.position = "none")
}

set.seed(GLOBAL_SEED)
p_dist_100 <- plot_wealth_distribution(100)
p_dist_1000 <- plot_wealth_distribution(1000)
p_dist_10000 <- plot_wealth_distribution(10000)

(p_dist_100 | p_dist_1000 | p_dist_10000) +
  plot_annotation(
```

```

title = "Final Wealth Distributions Across Kelly Strategies",
subtitle = "Dashed lines show median values",
theme = theme(plot.title = element_text(size = 14, face = "bold", hjust = 0.5),
              plot.subtitle = element_text(hjust = 0.5))
) +
plot_layout(guides = "collect") &
theme(legend.position = "bottom")

```

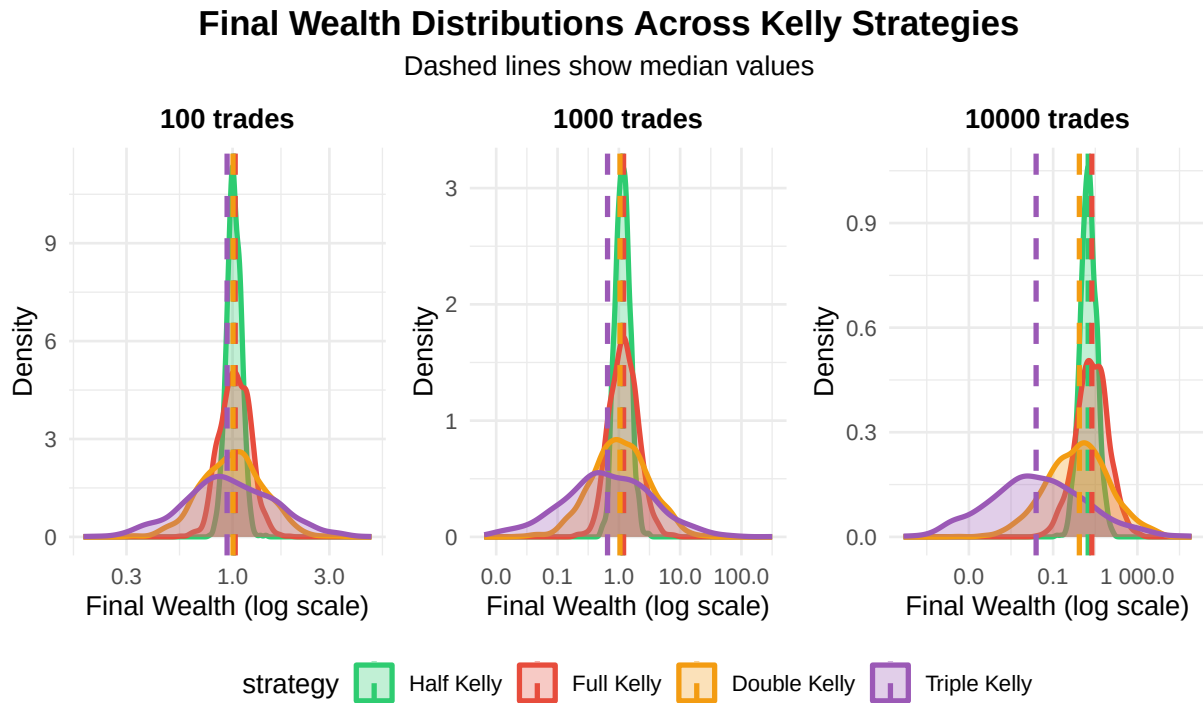


Figure A2: Final wealth distributions across 100, 1,000, and 10,000 simulated trades. Dashed lines show medians.

5 Figure A3 — Banca Intesa Cumulative Wealth (§3.2)

Cumulative wealth paths for Buy-and-Hold and three Kelly multiples on Banca Intesa (ISP.MI, 2007–2018). The Kelly fraction is estimated once from the full sample as $f^* = (\hat{\mu} - r_f) / \hat{\sigma}^2$ and applied as a static fraction of wealth allocated to the risky asset. Half and Full Kelly visibly dampen the 2008 and 2011 drawdowns; Triple Kelly tracks Buy-and-Hold's drawdown more closely but recovers a positive terminal wealth. The numerical metrics behind this figure are summarized in §3.2 of the main report.

```
prices_bi_xts <- getSymbols("ISP.MI",
                           from = "2007-01-01", to = "2018-12-31",
                           auto.assign = FALSE, warnings = FALSE)
prices_bi <- Ad(prices_bi_xts); colnames(prices_bi) <- "Price"

returns_bi <- na.omit(diff(prices_bi) / lag(prices_bi))[-1]
colnames(returns_bi) <- "Return"

returns_bi_df <- data.frame(date = index(returns_bi),
                           return = as.numeric(returns_bi))

mu_bi <- mean(returns_bi_df$return)
sigma2_bi <- var(returns_bi_df$return)
trading_days_per_year <- 261
rf_daily_bi <- 0.01 / trading_days_per_year

# Kelly fraction (Eq. 5 of CC20): f* = (mu - r) / sigma^2
f_star_bi <- (mu_bi - rf_daily_bi) / sigma2_bi

strategies_bi <- list(
  "Buy and Hold" = NA,
  "Half Kelly"   = 0.5 * f_star_bi,
  "Full Kelly"   = f_star_bi,
  "Triple Kelly" = 3.0 * f_star_bi
)

n_obs_bi <- nrow(returns_bi_df)
wealth_paths_bi <- matrix(NA, nrow = n_obs_bi + 1, ncol = length(strategies_bi))
colnames(wealth_paths_bi) <- names(strategies_bi)
wealth_paths_bi[1, ] <- 1.0

# Wealth update (Eq. 6 of CC20)
for (i in 1:n_obs_bi) {
  r_t <- returns_bi_df$return[i]
  for (j in seq_along(strategies_bi)) {
    f <- strategies_bi[[j]]; W_t <- wealth_paths_bi[i, j]
    if (names(strategies_bi)[j] == "Buy and Hold") {
      wealth_paths_bi[i + 1, j] <- W_t * (1 + r_t)
    }
  }
}
```



```

    } else {
      wealth_paths_bi[i + 1, j] <-
        (W_t - W_t * f) * (1 + rf_daily_bi) + (W_t * f) * (1 + r_t)
    }
  }
}

dates_bi <- c(index(prices_bi)[1], returns_bi_df$date)
wealth_bi_df <- as_tibble(wealth_paths_bi) %>%
  mutate(date = dates_bi) %>%
  pivot_longer(-date, names_to = "Strategy", values_to = "Wealth") %>%
  mutate(Strategy = factor(Strategy, levels = names(strategies_bi)))

```

```

ggplot(wealth_bi_df, aes(x = date, y = Wealth, color = Strategy)) +
  geom_line(linewidth = 0.7) +
  scale_y_continuous(labels = comma, breaks = pretty_breaks(n = 10)) +
  scale_x_date(date_labels = "%Y", date_breaks = "2 years") +
  scale_color_manual(values = c("Buy and Hold" = "#000000",
                                "Half Kelly" = "#0066CC",
                                "Full Kelly" = "#CC0000",
                                "Triple Kelly" = "#00CC66")) +
  labs(title = "Cumulative Returns for one Stock", x = "year", y = "wealth") +
  theme(legend.position = c(0.15, 0.95),
        legend.title = element_blank(),
        legend.background = element_rect(fill = "white", color = NA),
        panel.grid.minor = element_blank(),
        plot.title = element_text(hjust = 0.5))

```

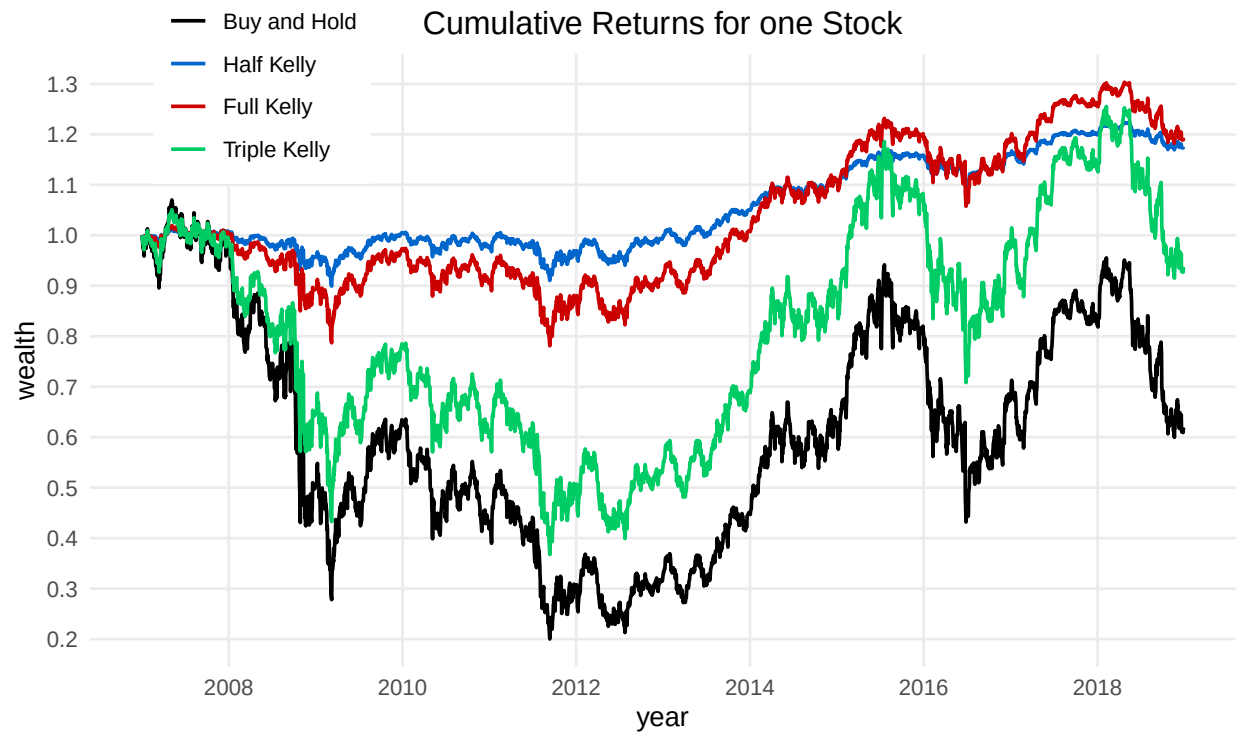


Figure A3: Cumulative wealth paths on Banca Intesa (ISP.MI), 2007-2018, for Buy-and-Hold and Half / Full / Triple Kelly.

6 Figure A4 — Mean-Variance Efficient Frontier, In-Sample EuroStoxx 50 (§3.3)

The mean-variance plane for the in-sample EuroStoxx 50 panel (2000–2018, monthly returns), with the four optimal portfolios overlaid: equal-weight, minimum-variance, tangent (max-Sharpe), and Kelly. The Kelly portfolio sits above the tangent point on the efficient frontier — the corner solution discussed in §3.3 of the main report (62.3% ITX.MC, 37.7% VIV.PA), achieved by maximizing the unconstrained quadratic form without the budget cap that binds the OOS variants. The frontier is computed by solving the standard QP at 100 target-return levels.

```
eurostoxx_tickers <- c(
  "AD.AS", "AI.PA", "ALV.DE", "AXA.PA", "ISP.MI", "BAS.DE", "BAYN.DE", "BBVA.MC",
  "BMW.DE", "BNP.PA", "CA.PA", "CRH.IR", "DBK.DE", "DTE.DE", "ENEL.MI", "ENGI.PA",
  "ENI.MI", "EOAN.DE", "G.MI", "IBE.MC", "KER.PA", "LVMH.PA", "OR.PA", "ORA.PA",
  "PHIA.AS", "SAF.PA", "SAN.PA", "SAN.MC", "SAP.DE", "SU.PA", "SIE.DE", "FP.PA",
  "TEF.MC", "URW.AS", "UNA.AS", "VIV.PA", "DG.PA", "BN.PA", "ITX.MC", "MUV2.DE",
  "NOKIA.HE", "VOW3.DE"
)

safe_download <- function(ticker, start, end) {
  tryCatch({
    d <- getSymbols(ticker, src = "yahoo", from = start, to = end,
                    auto.assign = FALSE, warnings = FALSE)
    if (!is.null(d) && nrow(d) > 0) {
      adj <- Adj(d); colnames(adj) <- ticker; adj
    } else NULL
  }, error = function(e) NULL)
}

price_data_list <- list()
for (ticker in eurostoxx_tickers) {
  d <- safe_download(ticker, as.Date("2000-01-01"), as.Date("2018-12-31"))
  if (!is.null(d)) price_data_list[[ticker]] <- d
  Sys.sleep(0.5)
}

prices_es <- do.call(merge, price_data_list)
prices_es <- prices_es[, colMeans(is.na(prices_es)) < 0.20]
prices_es <- na.locf(prices_es, na.rm = FALSE)
prices_es <- prices_es[complete.cases(prices_es), ]

prices_monthly <- to.monthly(prices_es, OHLC = FALSE)
returns_monthly <- na.omit(diff(log(prices_monthly)))

returns_df_es <- data.frame(
  DATE = index(returns_monthly),
  coredata(returns_monthly)
)
colnames(returns_df_es) <- c("DATE", colnames(returns_monthly))
```

```

# Long-only Kelly: maximize  $(\mu - r)'w - 0.5 w'Sigma w$ ,  $\sum(w) = 1$ 
optimize_kelly_constrained <- function(returns_matrix, risk_free_rate = 0.01) {
  ret_mat <- as.matrix(returns_matrix[, colnames(returns_matrix) != "DATE"])
  n <- ncol(ret_mat); mu <- colMeans(ret_mat, na.rm = TRUE)
  S <- cov(ret_mat, use = "complete.obs")

  eig_min <- min(eigen(S, only.values = TRUE)$values)
  if (eig_min < 1e-5) S <- S + diag(max(1e-4, abs(eig_min)*100)*mean(diag(S)), n)
  S <- (S + t(S)) / 2
  Dmat <- as.matrix(nearPD(S, corr = FALSE, keepDiag = TRUE)$mat)
  dvec <- mu - risk_free_rate / 12

  w <- solve.QP(Dmat, dvec,
                cbind(rep(1, n), diag(n)), c(1, rep(0, n)), meq = 1)$solution
  names(w) <- colnames(ret_mat)
  pr <- sum(w * mu); psd <- sqrt(as.numeric(t(w) %*% S %*% w))
  list(weights = w, expected_return = pr, volatility = psd,
        annualized_return = pr * 12, annualized_volatility = psd * sqrt(12),
        n_assets = sum(w > 0.001), mean_returns = mu)
}

# Tangent (max Sharpe, long-only)
optimize_tangent <- function(returns_matrix, risk_free_rate = 0.01) {
  ret_mat <- as.matrix(returns_matrix[, colnames(returns_matrix) != "DATE"])
  n <- ncol(ret_mat); mu <- colMeans(ret_mat, na.rm = TRUE)
  S <- cov(ret_mat, use = "complete.obs")
  excess <- mu - risk_free_rate / 12

  eig_min <- min(eigen(S, only.values = TRUE)$values)
  if (eig_min < 1e-6) S <- S + diag(max(1e-5, abs(eig_min)*10)*mean(diag(S)), n)
  S <- (S + t(S)) / 2
  S <- as.matrix(nearPD(S, corr = FALSE, keepDiag = TRUE)$mat)

  sol <- tryCatch(solve.QP(2 * S, rep(0, n),
                          cbind(excess, diag(n)), c(1, rep(0, n)), meq = 1),
                  error = function(e) {
                    list(solution = pmax(as.vector(solve(S) %*% excess), 0))
                  })
  w <- sol$solution / sum(sol$solution); names(w) <- colnames(ret_mat)
  pr <- sum(w * mu); psd <- sqrt(as.numeric(t(w) %*% S %*% w))
  list(weights = w, expected_return = pr, volatility = psd,
        annualized_return = pr * 12, annualized_volatility = psd * sqrt(12),
        n_assets = sum(w > 0.001), mean_returns = mu)
}

# Minimum variance, long-only, fully invested
optimize_min_variance <- function(returns_matrix) {

```

```

ret_mat <- as.matrix(returns_matrix[, colnames(returns_matrix) != "DATE"])
n <- ncol(ret_mat); mu <- colMeans(ret_mat, na.rm = TRUE)
S <- cov(ret_mat, use = "complete.obs")

eig_min <- min(eigen(S, only.values = TRUE)$values)
if (eig_min < 1e-6) S <- S + diag(max(1e-5, abs(eig_min)*10)*mean(diag(S)), n)
S <- (S + t(S)) / 2

w <- solve.QP(2 * S, rep(0, n),
              cbind(rep(1, n), diag(n)), c(1, rep(0, n)), meq = 1)$solution
names(w) <- colnames(ret_mat)
pr <- sum(w * mu); psd <- sqrt(as.numeric(t(w) %*% S %*% w))
list(weights = w, expected_return = pr, volatility = psd,
      annualized_return = pr * 12, annualized_volatility = psd * sqrt(12),
      n_assets = sum(w > 0.001), mean_returns = mu)
}

# Equal-weight benchmark
create_equal_weight <- function(returns_matrix) {
  ret_mat <- as.matrix(returns_matrix[, colnames(returns_matrix) != "DATE"])
  n <- ncol(ret_mat); w <- rep(1/n, n); names(w) <- colnames(ret_mat)
  mu <- colMeans(ret_mat, na.rm = TRUE)
  S <- cov(ret_mat, use = "complete.obs")
  pr <- sum(w * mu); psd <- sqrt(as.numeric(t(w) %*% S %*% w))
  list(weights = w, expected_return = pr, volatility = psd,
        annualized_return = pr * 12, annualized_volatility = psd * sqrt(12),
        n_assets = n, mean_returns = mu)
}

kelly <- optimize_kelly_constrained(returns_df_es, risk_free_rate = 0.01)
tangent <- optimize_tangent(returns_df_es, risk_free_rate = 0.01)
minvar <- optimize_min_variance(returns_df_es)
eqwt <- create_equal_weight(returns_df_es)

```

```

compute_efficient_frontier <- function(returns_matrix, n_points = 100) {
  ret_mat <- as.matrix(returns_matrix[, colnames(returns_matrix) != "DATE"])
  n <- ncol(ret_mat); mu <- colMeans(ret_mat, na.rm = TRUE)
  S <- cov(ret_mat, use = "complete.obs")

  eig_min <- min(eigen(S, only.values = TRUE)$values)
  if (eig_min < 1e-6) S <- S + diag(max(1e-5, abs(eig_min)*10)*mean(diag(S)), n)
  S <- (S + t(S)) / 2

  sol_mv <- solve.QP(2*S, rep(0, n),
                    cbind(rep(1, n), diag(n)), c(1, rep(0, n)), meq = 1)
  w_mv <- sol_mv$solution
  ret_mv <- sum(w_mv * mu); vol_mv <- sqrt(as.numeric(t(w_mv) %*% S %*% w_mv))
}

```

```

targets <- seq(ret_mv, max(ret_mv * 2, 0.02), length.out = n_points)
frontier <- map_dfr(targets, function(tr) {
  sol <- tryCatch(solve.QP(2*S, rep(0, n),
                          cbind(mu, rep(1, n), diag(n)),
                          c(tr, 1, rep(0, n)), meq = 2),
                  error = function(e) NULL)
  if (is.null(sol)) return(NULL)
  w <- sol$solution
  tibble(return_monthly = sum(w * mu),
          volatility_monthly = sqrt(as.numeric(t(w) %*% S %*% w)))
}) %>% bind_rows()

list(frontier = frontier, minvar_return = ret_mv, minvar_volatility = vol_mv)
}

frontier_res <- compute_efficient_frontier(returns_df_es, n_points = 100)

portfolio_pts <- tibble(
  Portfolio = c("Equal Weight", "Min Variance", "Tangent Pf", "Kelly Pf"),
  Return = c(eqwt$expected_return, frontier_res$minvar_return,
             tangent$expected_return, kelly$expected_return) * 100,
  Risk = c(eqwt$volatility, frontier_res$minvar_volatility,
           tangent$volatility, kelly$volatility) * 100
)

rf_m <- 0.01 / 12 * 100
cml_slope <- (tangent$expected_return * 100 - rf_m) / (tangent$volatility * 100)
cml <- tibble(risk = seq(0, 8.5, length.out = 100),
              return = rf_m + cml_slope * seq(0, 8.5, length.out = 100))

ggplot() +
  geom_line(data = frontier_res$frontier,
            aes(x = volatility_monthly * 100, y = return_monthly * 100),
            color = "gray60", linewidth = 1) +
  geom_line(data = cml[cml$risk <= 8.5 & cml$return <= 2.0, ],
            aes(x = risk, y = return),
            color = "gray40", linewidth = 0.8, alpha = 0.7) +
  geom_point(data = portfolio_pts,
             aes(x = Risk, y = Return, color = Portfolio, shape = Portfolio),
             size = 5) +
  scale_color_manual(values = c("Equal Weight" = "black",
                                "Min Variance" = "#F39C12",
                                "Tangent Pf" = "#3498DB",
                                "Kelly Pf" = "#E74C3C")) +
  scale_shape_manual(values = c("Equal Weight" = 15, "Min Variance" = 16,
                                "Tangent Pf" = 18, "Kelly Pf" = 15)) +
  scale_x_continuous(limits = c(3, 8.5), breaks = seq(3, 8, 1)) +

```

```

scale_y_continuous(limits = c(0, 1.8), breaks = seq(0, 1.8, 0.3)) +
labs(title = "Mean-Variance Space", x = "Risk", y = "Return",
     color = "", shape = "") +
theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 15),
      legend.position = c(0.85, 0.25),
      legend.background = element_rect(fill = "white", color = "black",
                                       linewidth = 0.4),
      panel.background = element_rect(fill = "white"),
      panel.border = element_rect(color = "black", fill = NA,
                                  linewidth = 0.6))

```

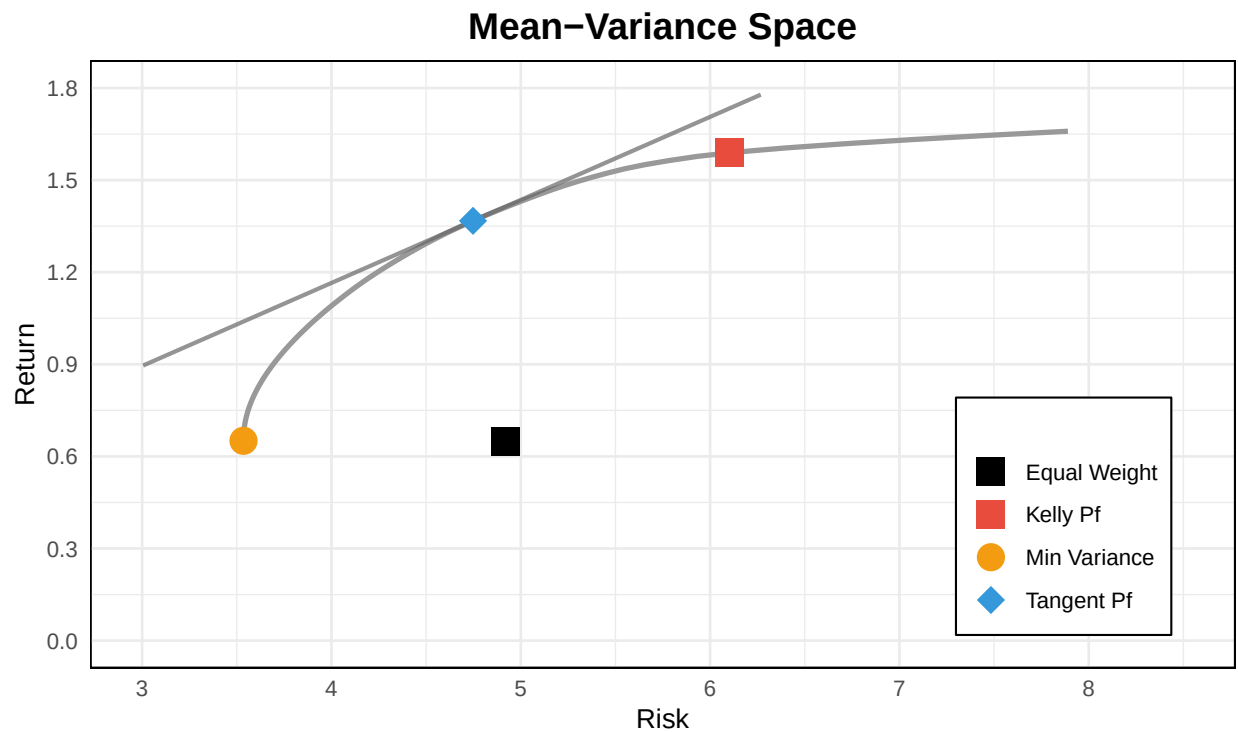


Figure A4: Mean-variance efficient frontier with optimal portfolios, in-sample EuroStoxx 50 (2000-2018, monthly).

7 Figure A5 — Long-Only OOS Cumulative Returns, EuroStoxx 50 (§3.4)

Out-of-sample cumulative wealth for the five long-only strategies on EuroStoxx 50 (2005–2018), produced by a rolling-window backtest with 756-day window and monthly re-optimization. The visual signature of the constraint collapse documented in §3.4 is direct: **Full Kelly, Double Kelly, and Triple Kelly produce identical daily returns and therefore plot as a single line** — they share the same QP solution because the long-only budget cap $\sum w_i = f \leq 1$ binds for any $f \geq 1$, so all three strategies collapse to the same $f = 1$ corner solution. The legend lists five entries but only three distinct trajectories are visible: Half Kelly (lower red curve), MinVar (lower green curve), and the collapsed-aggressive group (the upper trajectory, drawn in Triple Kelly’s color since it is plotted last and occludes the underlying Full and Double Kelly lines). The numerical performance summary (Sharpe, max drawdown, annual return) is reported in Table 2 of the main report.

```
return_xts_oos <- na.omit(diff(log(prices_es)))
price_xts_oos  <- prices_es[index(return_xts_oos)]

stopifnot(ncol(return_xts_oos) > 1, nrow(return_xts_oos) > 252 * 3)

rf_daily_oos <- 0.01 / 252

dataset_list_oos <- list("EUROSTOXX" =
  list(prices = price_xts_oos,
        returns = return_xts_oos))

safe_kelly_fraction <- function(x, rf) {
  mu <- mean(x, na.rm = TRUE); sigma2 <- var(x, na.rm = TRUE)
  if (is.na(mu) || is.na(sigma2) || sigma2 == 0) return(0)
  (mu - rf) / sigma2
}

# Long-only Kelly factory: sum(w) = f in [0, 1], w_i >= 0
kelly_strategy_factory_long <- function(multiplier = 1) {
  function(dataset, ...) {
    ret <- dataset$returns
    if (ncol(ret) < 1) stop("returns matrix malformed")

    mu    <- colMeans(ret, na.rm = TRUE)
    Sigma <- cov(ret, use = "pairwise.complete.obs")
    n     <- length(mu)

    f <- max(min(multiplier, 1), 0)

    w <- solve.QP(Sigma, mu - rf_daily_oos,
                  cbind(rep(1, n), diag(1, n)),
                  c(f, rep(0, n)), meq = 1)$solution
```



```

    names(w) <- colnames(ret); w
  }
}

min_var_weights <- function(returns) {
  S <- cov(returns, use = "pairwise.complete.obs"); n <- ncol(returns)
  if (det(S) <= 1e-8) S <- S + diag(1e-6, n)
  w <- solve.QP(2 * S, rep(0, n),
               cbind(rep(1, n), -rep(1, n), diag(n)),
               c(1, -1, rep(0, n)), meq = 1)$solution
  w / sum(w)
}

markowitz_minvar <- function(dataset, ...) {
  ret <- dataset$returns
  w <- tryCatch(min_var_weights(ret),
               error = function(e) rep(1/ncol(ret), ncol(ret)))
  if (length(w) != ncol(ret) || any(!is.finite(w))) w <- rep(1/ncol(ret), ncol(ret))
  names(w) <- colnames(ret); w
}

kelly_half_long <- kelly_strategy_factory_long(0.5)
kelly_full_long <- kelly_strategy_factory_long(1.0)
kelly_double_long <- kelly_strategy_factory_long(2.0)
kelly_triple_long <- kelly_strategy_factory_long(3.0)

portfolio_list_long <- list(
  "Half Kelly" = kelly_half_long,
  "Full Kelly" = kelly_full_long,
  "MinVar" = markowitz_minvar,
  "Double Kelly" = kelly_double_long,
  "Triple Kelly" = kelly_triple_long
)

bt_long <- portfolioBacktest::portfolioBacktest(
  portfolio_list_long, dataset_list_oos,
  lookback = 252 * 3, optimize_every = 21, rebalance_every = 21
)

bt_long_summary <- backtestSummary(bt_long)

print(backtestChartCumReturn(bt_long))

```

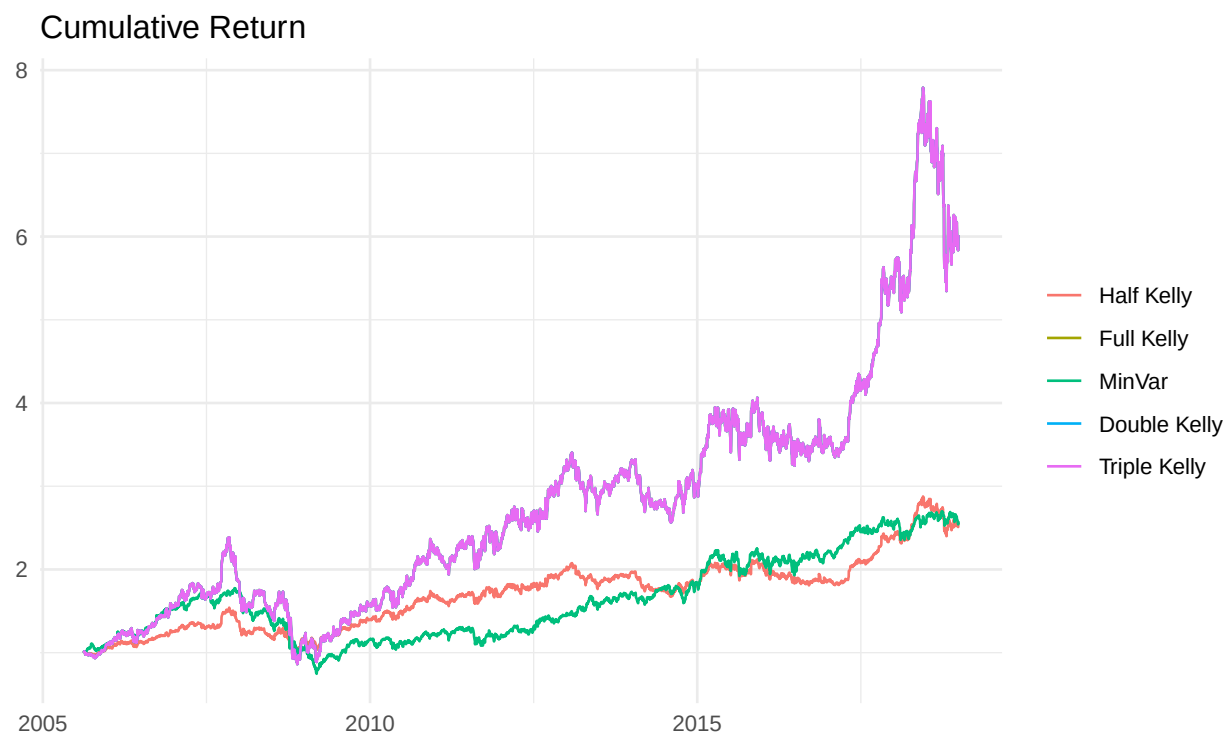


Figure A5: Long-only OOS cumulative returns, EuroStoxx 50, 2005-2018. Full Kelly, Double Kelly, and Triple Kelly are plotted as separate series but trace identical paths — the long-only budget cap collapses them onto a single QP solution, and Triple Kelly's line (drawn last) occludes the other two.

8 Figure A6 — HAC Pairwise Return Differences (§3.5)

This is the project’s central inferential extension. For each pair of strategies (A, B) we form the daily return-difference series $d_t = r_t^A - r_t^B$, regress on a constant, and plot the mean and 95% confidence interval using Newey-West HAC standard errors at five lags. The pointrange visualization makes the inferential picture immediate: only Half Kelly vs Full Kelly excludes zero, and the gap is approximately 4 basis points per day — driven entirely by the constraint collapse. Half-vs-MinVar and Full-vs-MinVar both straddle zero, consistent with the p -values in Table 3 of the main report (0.86 and 0.14, respectively).

```
# Extract daily strategy returns from the long-only backtest
get_ret <- function(name) as.numeric(bt_long[[name]][[1]]$return)
rets <- data.frame(
  "Half Kelly" = get_ret("Half Kelly"),
  "Full Kelly" = get_ret("Full Kelly"),
  "MinVar"     = get_ret("MinVar"),
  check.names = FALSE
)

pairs <- list(c("Half Kelly", "Full Kelly"),
             c("Half Kelly", "MinVar"),
             c("Full Kelly", "MinVar"))

# HAC point-and-interval estimates for each pair
ci_df <- do.call(rbind, lapply(pairs, function(p) {
  d <- rets[[p[1]]] - rets[[p[2]]]
  fit <- lm(d ~ 1)
  ct <- coeftest(fit, vcov = NeweyWest(fit, lag = 5, prewhite = FALSE))
  data.frame(Pair = paste(p, collapse = " vs "),
             Mean = ct[1, 1],
             Lower = ct[1, 1] - 1.96 * ct[1, 2],
             Upper = ct[1, 1] + 1.96 * ct[1, 2])
}))

ggplot(ci_df, aes(x = Pair, y = Mean)) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "grey50") +
  geom_pointrange(aes(ymin = Lower, ymax = Upper),
                 size = 0.8, linewidth = 0.9, color = "steelblue") +
  labs(title = "Mean Daily Return Differences (HAC 95% CI)",
       y = "Mean Difference", x = NULL) +
  theme_minimal() +
  theme(axis.text.x = element_text(size = 10))
```

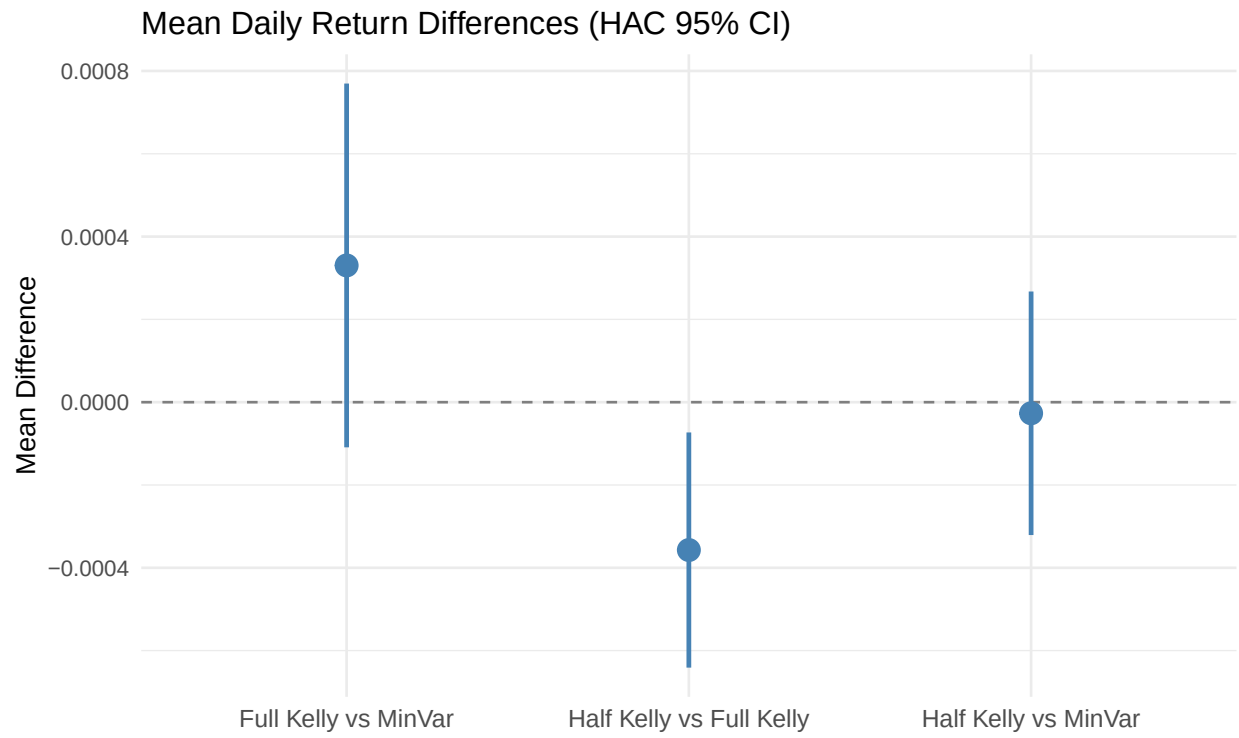


Figure A6: Mean daily return differences with 95% HAC confidence intervals, long-only OOS.

9 Figure A7 — Unconstrained Allocation Stack, Half Kelly (§3.6)

Asset weights through time for the Half Kelly variant under the unconstrained QP (shorts permitted, leverage capped at $|f| \leq 3$). The plot is the visual evidence for the over-leverage pathology described in §3.6: the optimizer holds aggregate gross leverage near the +3 ceiling for sustained periods, inverts to a coordinated short position across most assets through 2009–2010 (visible as the stack going negative), and never settles into a stable allocation. The unconstrained backtest confirms that the long-only constraint in the main pipeline is doing real work — removing it lets the optimizer chase sample-mean signal that does not survive out-of-sample.

```
# Unconstrained Kelly: equal-weight base scaled by the Kelly scalar,
# leverage capped at +-3 for numerical stability
kelly_strategy_factory_uncon <- function(multiplier) {
  function(dataset, ...) {
    ret <- dataset$returns
    w_base <- rep(1/ncol(ret), ncol(ret))
    port_ret <- rowMeans(ret, na.rm = TRUE)
    f <- safe_kelly_fraction(port_ret, rf_daily_oos) * multiplier
    f <- max(min(f, 3), -3)
    w <- w_base * f; names(w) <- colnames(ret); w
  }
}
```

```
kelly_half_uncon <- kelly_strategy_factory_uncon(0.5)
kelly_full_uncon <- kelly_strategy_factory_uncon(1.0)
kelly_double_uncon <- kelly_strategy_factory_uncon(2.0)
kelly_triple_uncon <- kelly_strategy_factory_uncon(3.0)
```

```
portfolio_list_uncon <- list(
  "Half Kelly" = kelly_half_uncon,
  "Full Kelly" = kelly_full_uncon,
  "Double Kelly" = kelly_double_uncon,
  "Triple Kelly" = kelly_triple_uncon
)
```

```
bt_uncon <- portfolioBacktest::portfolioBacktest(
  portfolio_list_uncon, dataset_list_oos,
  lookback = 252 * 3, optimize_every = 21, rebalance_every = 21
)
```

```
print(backtestChartStackedBar(bt_uncon, portfolio = "Half Kelly", legend = TRUE))
```

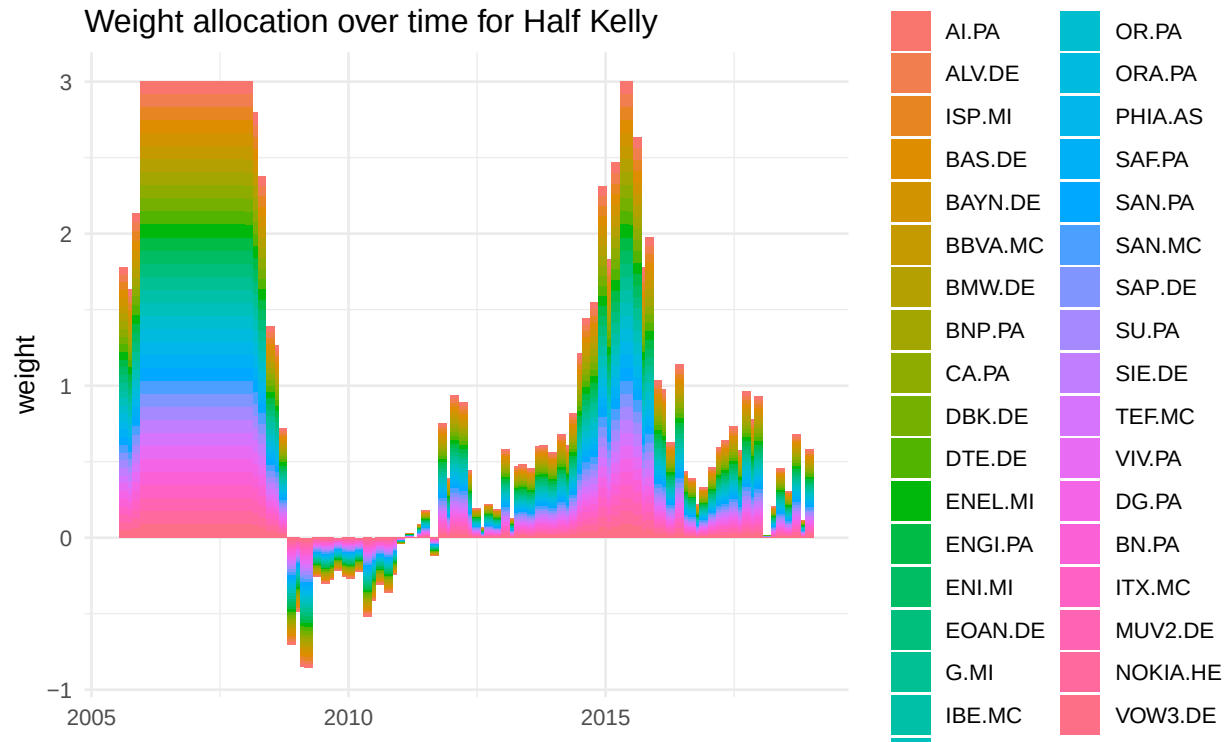


Figure A7: Unconstrained EuroStoxx 50 weights over time, Half Kelly. Aggregate weight exceeding ± 1 reflects leverage; negative segments reflect coordinated short positions.